

Vulnerability Assessment and Remediation Report

Part 2: AWS IAM Policy Security

System Architecture and Design

The AWS IAM agent is designed as an agentic system that balances LLM reasoning with deterministic code execution. To ensure auditability and reduce AI mistakes, the system follows a controlled analysis loop.

Agent Logic and Explanation

The design avoids a monolithic approach by separating the classification task from the remediation task.

- **Deterministic Analysis Tools:** Python functions are utilized to extract objective facts from the JSON policy, such as identifying wildcard actions, checking resource scopes, and detecting dangerous combinations like privilege escalation.
- **Agent Loop:** The LLM iterates through these tools. It is explicitly prohibited from calling *submit_verdict* until all four analysis tools have been executed, ensuring a comprehensive assessment.
- **Remediation Strategy:** To prevent "eager remediator" failure, the agent completes the full classification before initiating a second different LLM call to generate the remediated policy and change log.

Results

The system was evaluated 8 AWS IAM policies, ranging from critical security misconfigurations to hardened standards, as WEAK and STRONG.

Policy Category	AWS Accuracy
Weak Policies	100% (5/5 identified)
Strong Policies	100% (3/3 identified)
Total Agreement	100%

Part 3: Multi-Cloud IAM Security

System Architecture and Design

Part 3 extends the architecture to support multiple cloud providers (AWS and GCP) through the implementation of a Router pattern.

- **Cloud Router:** A main entry point that performs $O(1)$ detection of the cloud provider by inspecting JSON keys ("Statement" for AWS, "bindings" for GCP).
- **Extensibility:** This architecture is designed to be open for extension but closed for modification- adding a new provider requires only a new agent module without altering existing code.

Classification Criteria and Logic

The GCP agent adheres to the same *run_agent()* interface as the AWS agent, ensuring uniform downstream handling with a tailored security criteria for the GCP IAM model:

- **WEAK Criteria:**
 - **G1:** Presence of primitive roles (owner, editor, viewer).
 - **G2:** Public access via `allUsers` or `allAuthenticatedUsers`.
 - **G3/G4:** Unconditioned sensitive roles (e.g., `serviceAccountTokenCreator`) or broad service-admin roles.
- **STRONG Criteria:** All bindings must use non-primitive roles, lack public members, include specific identifiers, and possess proper Condition blocks.

Results

The multi-cloud agent was evaluated 5 different GCP IAM policies.

All 5 policies were classified correctly.